Sustainable Digital (TI616I)

Deliverable Sustainable Digital (TI616I)

Rayan Boumedine / Alexis Launay / Ambroise Couturier / Paul Mahaut

---

Phase 1 — Product Scoping

1. Identify the Need and the Value Proposition

LeafTask is a specialized event and task management platform designed to address the challenge of users struggling to maintain eco-friendly habits within their daily lives and social circles. The website aims to solve the need for a practical tool that allows individuals, families, and couples to organize and assign tasks specifically geared toward adopting eco-responsible practices.

By providing a structured way to set up and send "eco-tasks," the platform brings unique value to its users, enabling them to track their progress while actively participating in a shared environmental initiative. This project is highly realistic for a team of 4 to 5 students as its functional scope aligns with the team's technical skill set and strictly adheres to the sustainable digital principles and development norms taught at EFREI. This focus on simplicity and utility ensures a lean, high-performance service that fulfills its core mission without the environmental overhead of traditional, feature-heavy management tools.ing our skill set and it follows the principles and norms learn in this course and at EFREI.

2. Define the MVP

The MVP focuses strictly on the "Must-Have" features required for a functional, usable product:

A secure User Registration/Login system and a primary Task Management system allowing for full CRUD (Create, Read, Update, Delete) operations.

This ensures that users can immediately start managing their "eco-tasks" through a lean interface that handles essential authentication and core business data via the User and Task entities.

While several advanced features have been identified, such as an Administrative Panel for system hygiene and specialized Task Categorization for better organization, these are officially postponed to Version 2.

By prioritizing these essential features and deferring secondary complexity, the team ensures the project remains realistic and consistent with Green IT principles.

This focused scope allows the team to deliver a high-quality, sustainable product that effectively solves the user's need while maintaining a reduction in CO_2 emissions.

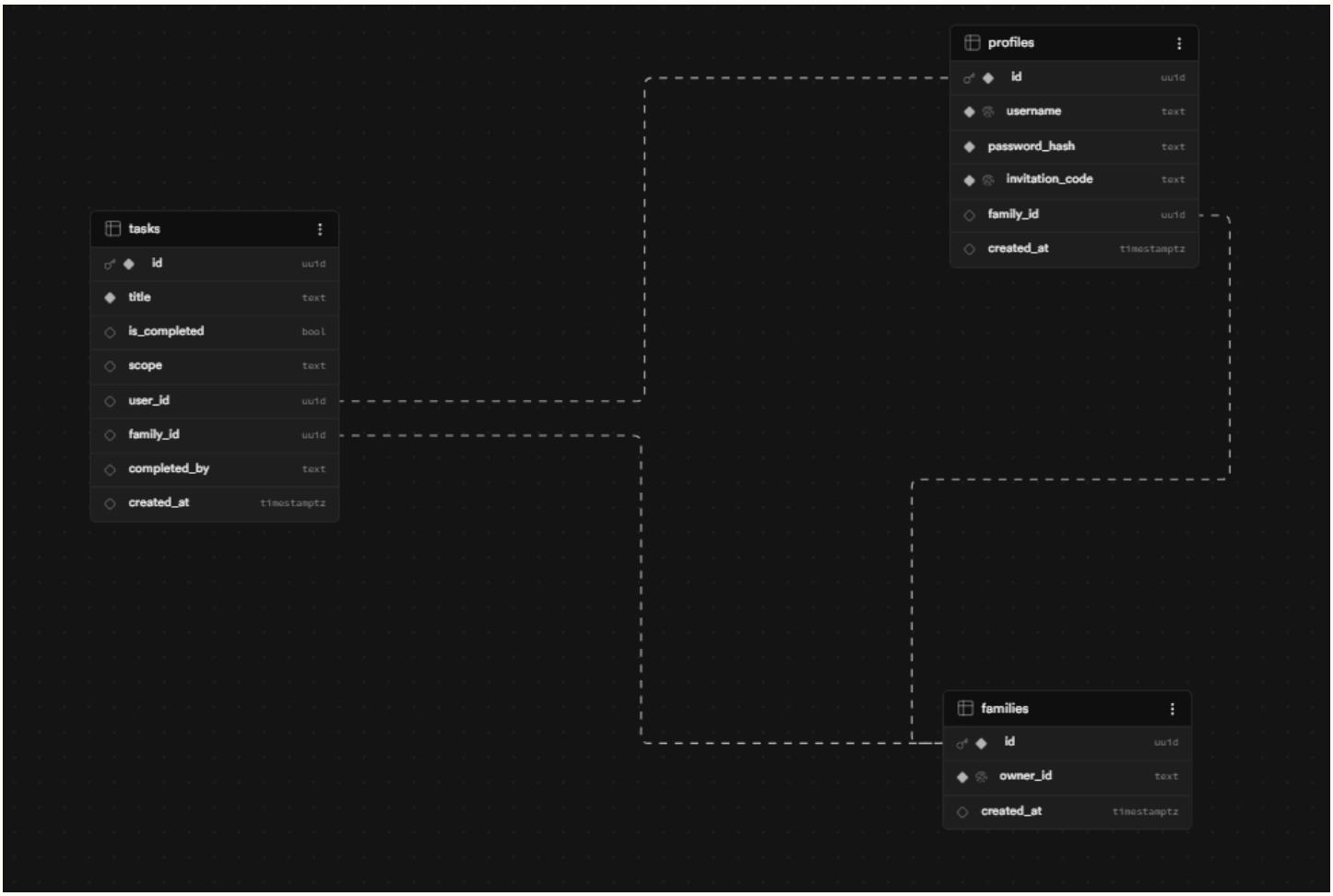
3. Define Users and Their Roles

This task manager is working as it follows. Your are a user either alone or taking part to a family. The user in charge of the family is the one giving his family code to the other. This user is the administrator of his family, he can promote or exclude users from his family. Each user is able to join a family, create a family, create his own task, delete his own task.

The family tasks and user task are two differents objects and are not stored in the same tables as long as they are not duplicated.

4. Identify the Data to Be Managed

We can see the data management as this table representing the database



5. Write User Stories

We decided to implement 3 differents user stories that will permits us to understand how every relations are working :

Using the mothodology learn in the previous classes we get

1. The Family Coordinator point of view

As a parent or household manager,

I want to create a unique family code and invite my family members to join my group,

so that I can delegate household chores and see exactly who has completed them.

2. The Collaborative Member point of view

As a family member or roommate,

I want to join a family group using a shared code and validate tasks assigned to the group,

so that the rest of the family knows the task is done and that I was the one who did it.

3. The Organized Individual point of view

As a user with both personal and shared responsibilities,

I want to have a private dashboard separate from the family dashboard,

so that I can track my own errands without cluttering the shared family list

6. Build a Mini Backlog

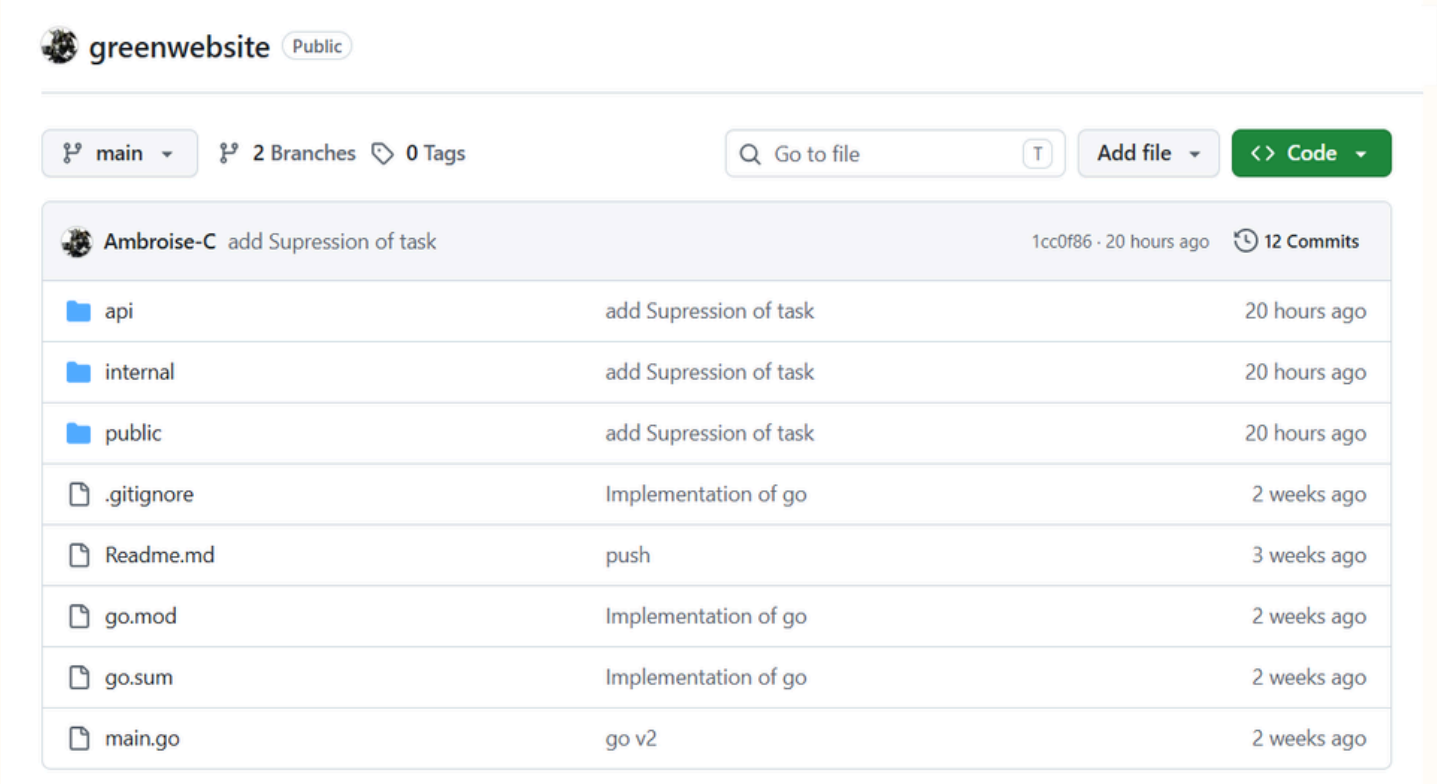
This back log is the one that we setted up for the V1 of our project. This is used so ew know the roadmap of the future implementations. At therm we want to present this roadmap with all the ETA at done proving our point with the feasability of our project.

Priority	Item	Description	ETA
P0 (Critique)	Login	Allow users to create an account and log in.	☑ DONE
P0 (Critique)	Families	Invitation code system and Leader/Members link.	☑ DONE
P1 (Important)	Double Dashboard	Visual separation between Private and Family tasks.	☑ DONE
P1 (Important)	Validation for tasks	Visual validation	☑ DONE
P2 (Optionnal)	Dark mode	CSS that permits to have a dark mode	⌛ Pending
P2 (Optionnal)	Database	Migrate from LocalStorage to a real SQL database.	📅 future

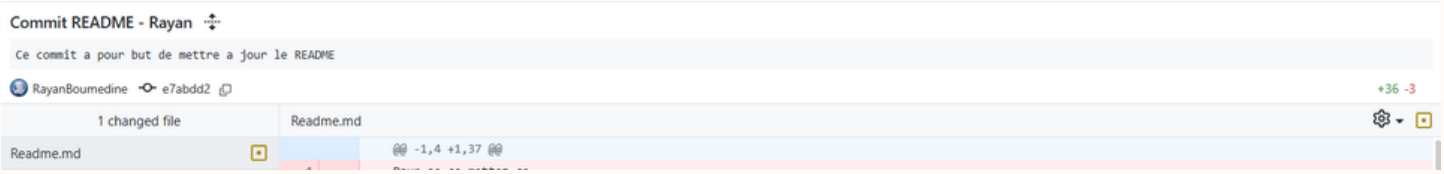
Phase 2 — Team Organization and Collaboration with GitHub

1. Create the Shared Repository

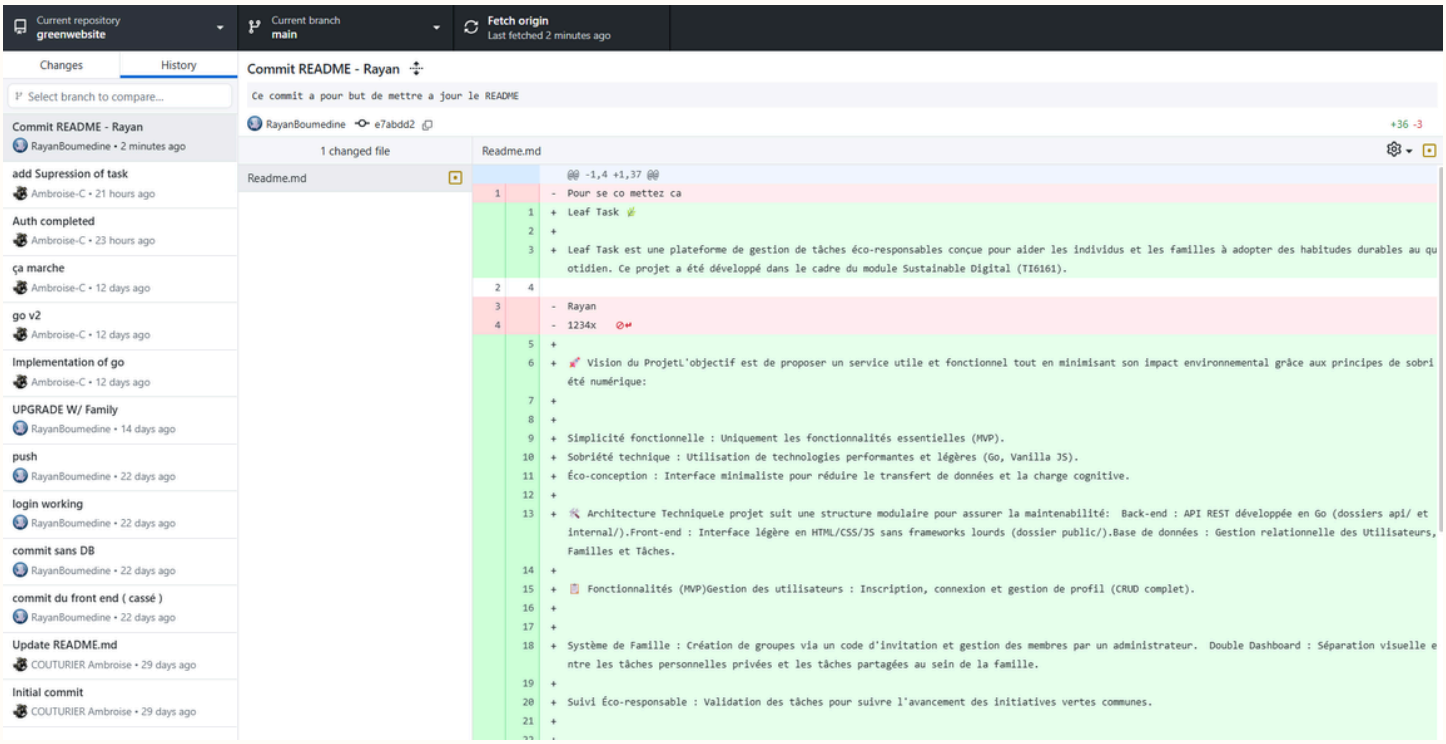
We first created our repository to start working on this project.



The collaborators which were Rayan, Paul, Alexis and Ambroise were added through the github system and we started to work over the project.



The shared work was divided through a series of commits that will permits us to distinguish what is done and what is left.



The main thing used in this project was github desktop which permitted us to have a clear visual of the differents steps of this project. Through the differents commits, we made sure that everything was done and that the construction of our project was following the path that we intended.

Phase 3 — Functional and Technical Design

1. Functional Architecture

Leaf Task follows a Client-Server architecture designed for maximum efficiency and low resource consumption.

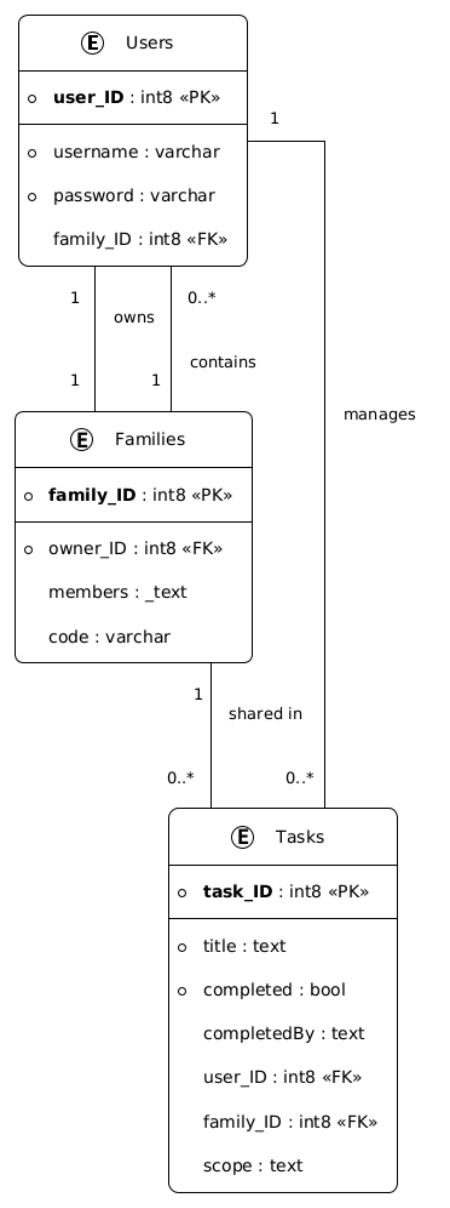
For the front-end, a lightweight approach using vanilla HTML5, CSS3, and JavaScript. It communicates with the server via asynchronous Fetch API calls to minimize page reloads. By avoiding heavy external libraries, we minimize the weight of the files sent over the network. Each kilobyte saved reduces the energy consumed by routers and data centers

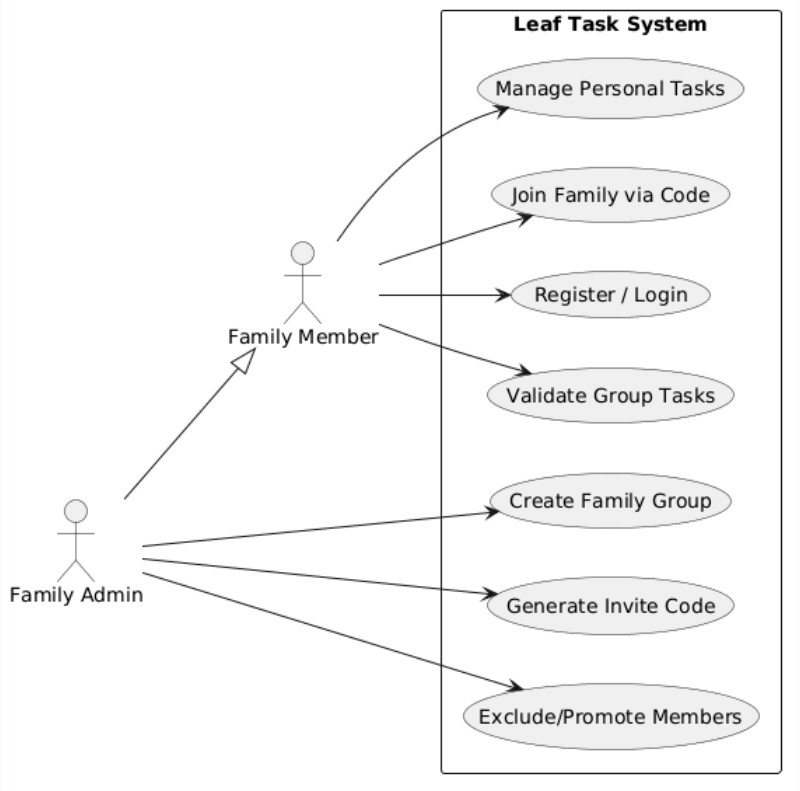
For the back-end, a high-performance REST API built with Go. It handles authentication, business logic for eco-tasks, and family group management. Go handles multiple users with "Goroutines", which are much lighter than traditional threads, allowing the server to stay responsive with minimal power.

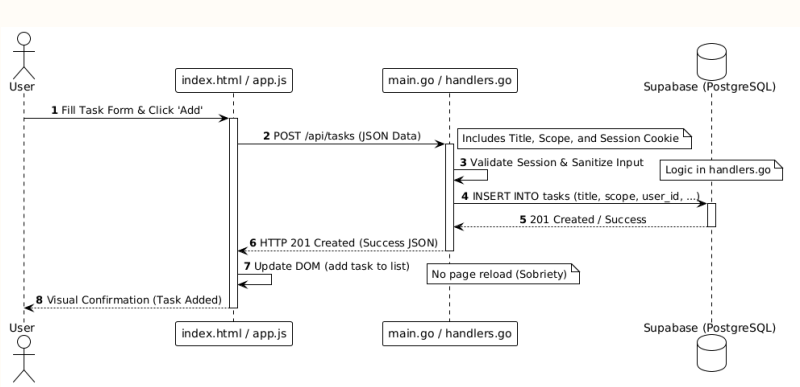
Finaly, for the database, we have a relational SQL database ensuring data integrity for users, families, and tasks. As a powerful relational database, Postgres allows us to use precise queries to fetch only the data we need, preventing unnecessary energy use during data transit. Moreover, we decided to host our database on Supabase. Supabase utilizes a shared infrastructure model. Instead of running a dedicated server 24/7 that wastes energy during idle times, resources are scaled and shared efficiently.

2. Diagrams

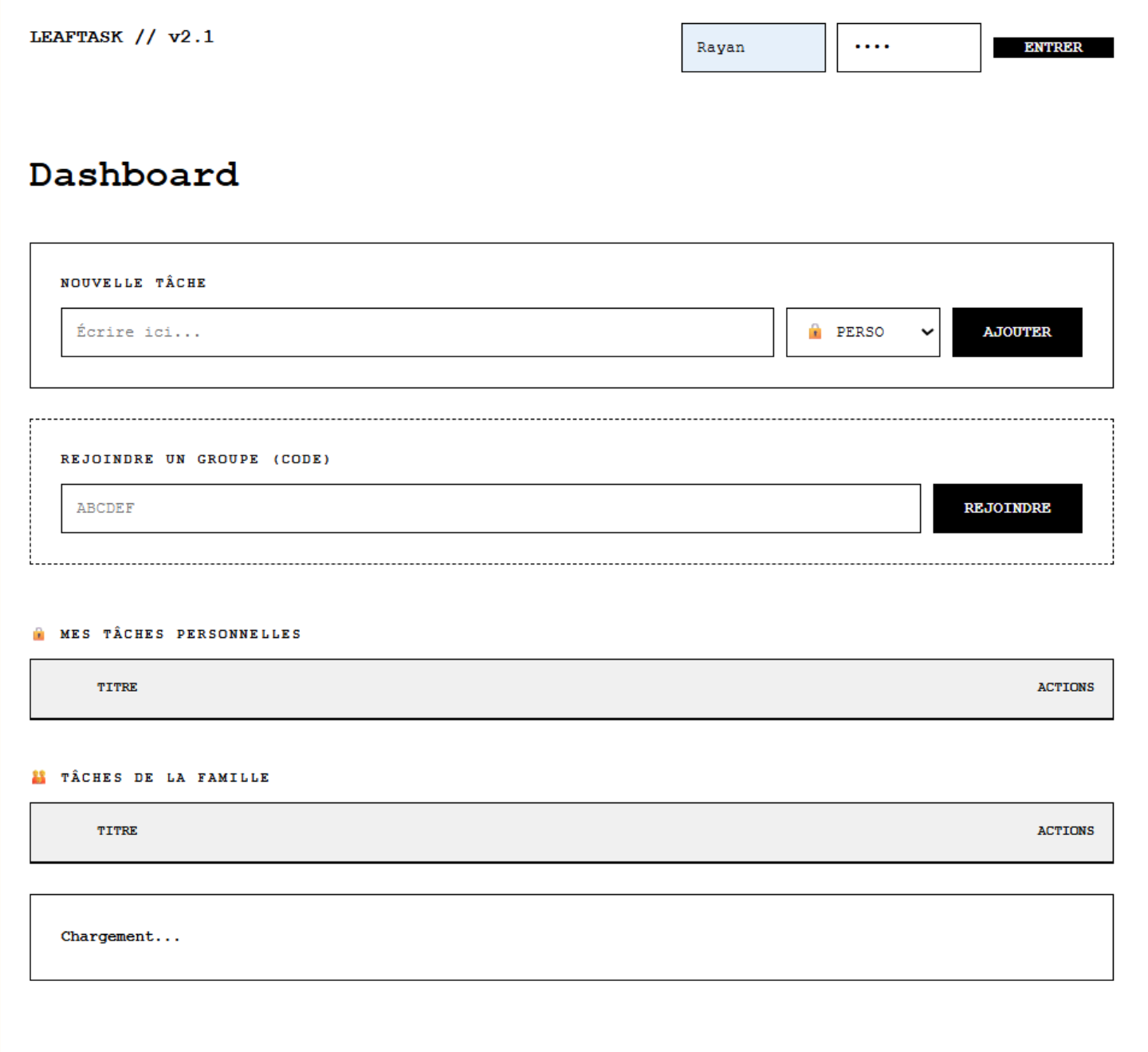
First, we implemented a Class diagram to have a clear understanding of how our supabase will manage the links between each entities. Then we worked over the use cased diagram that will give us a clear look of the boundaries of what a user can do. Finaly, we maid a sequence diagram to have a clear idea of how the front end will interact with the back end to add a task.







3.Interface and wireframe



Conclusion of the first part of the project

In conclusion, the first phase of the Leaf Task project has established a clear and realistic roadmap for a service that balances high performance with digital sobriety. By selecting Go, Vanilla JS, and Supabase, we have created a technical architecture that minimizes energy consumption and data transfer while delivering a functional "Double Dashboard" for eco-friendly habits. Our collaborative workflow on GitHub and the definition of a strict MVP ensure that the project remains focused on utility rather than unnecessary features. This foundational work proves that a digital tool can be both professionally engineered and environmentally responsible, setting a strong stage for the final development of the platform.

Phase 4 — Implementation — Front-end, Back-end and Database

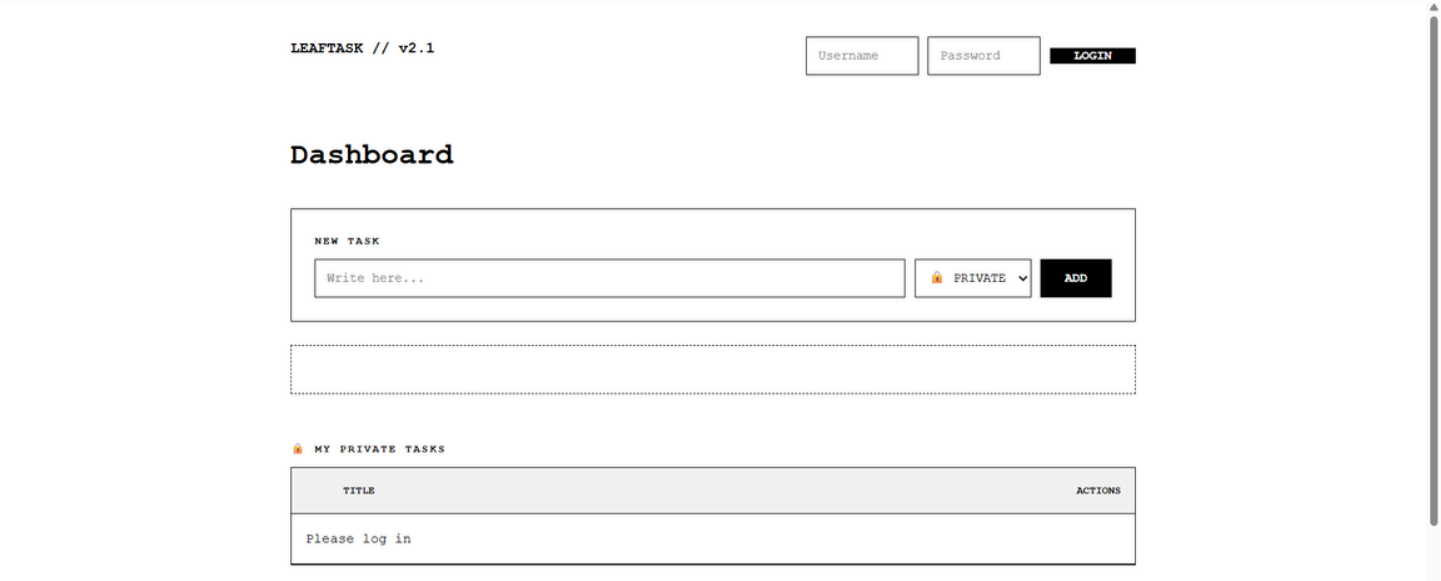
The first part of this second delivery is going to be focusing on the building of the app and what exacly have we done

1 — Justified Technology Choices

For a Green IT project, the choice of technology directly impacts energy consumption through network data transfer and client-side CPU processing.

Technology	Choice	Green IT Justification
Language	Vanilla JavaScript	Avoids the overhead of heavy frameworks like React or Vue, which can add 200–500 KB of JS bundles. This reduces the energy required for the browser to parse and execute code.
Styling	Native CSS3	We opted for a single, lightweight styles.css file. By avoiding CSS-in-JS or heavy frameworks like Bootstrap, we minimize "CSS bloat" and keep the DOM rendering path fast.
Typography	System Fonts	Using fonts already installed on the user's OS eliminates external HTTP requests to services like Google Fonts. This saves ~30–80 KB per page load.
Architecture	Single Page Application Logic	The render() function updates specific DOM elements dynamically. This prevents the server from re-sending the entire HTML structure on every interaction, significantly reducing data transfer.

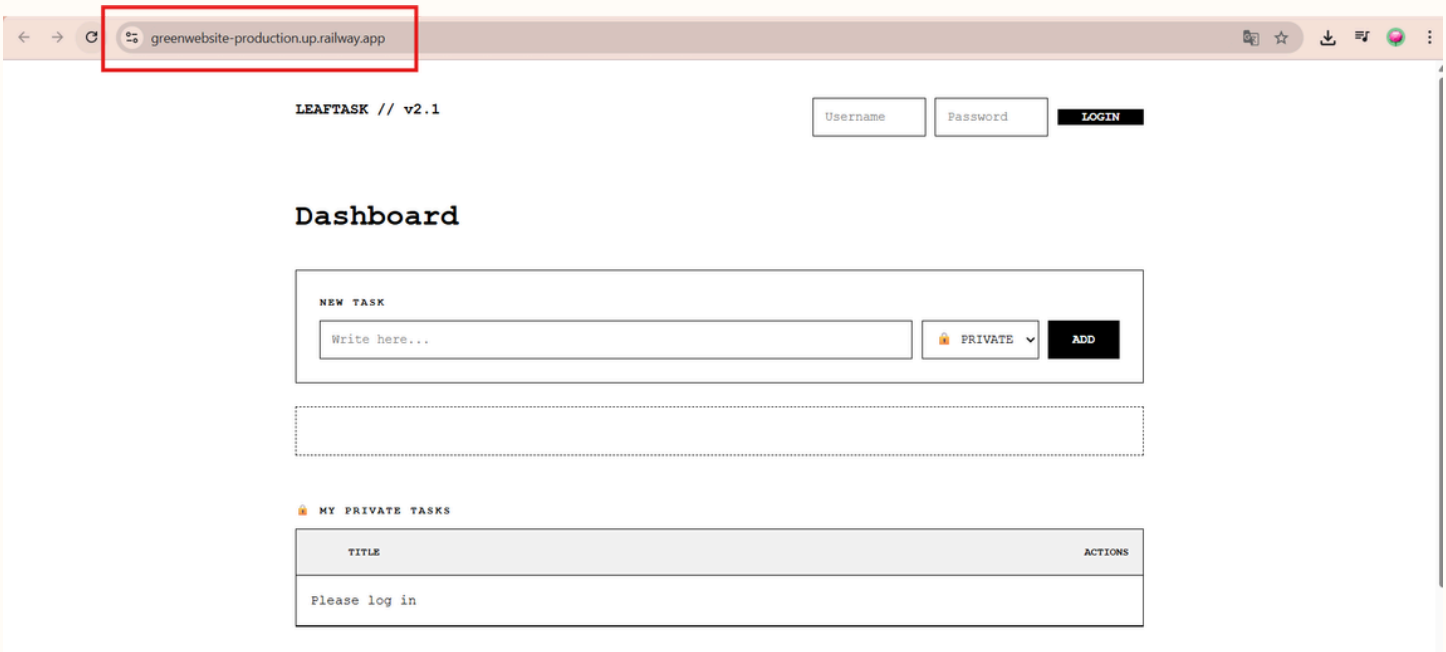
These technologies where used to have the following basis. We first want a first view of the app before doing a backend.



The back-end was designed to minimize server-side resource consumption (CPU/RAM) and optimize database interactions.

Technology	Choice	Green IT Justification
Language	Go (Golang)	Go is a compiled language that produces a single, lightweight binary. Its concurrency model (goroutines) is significantly less RAM-intensive than traditional threads, allowing it to handle many users with minimal energy consumption.
Framework	Standard Library	We avoided heavy frameworks. By using Go's standard library, we reduced the number of dependencies, which decreases the size of the deployed bundle and minimizes security risks.
Database	PostgreSQL (Supabase)	Using a robust relational database allows for precise SQL queries. This avoids loading unnecessary objects into memory, unlike some NoSQL approaches.
Hosting	Railway	Railway supports lightweight container-based deployments that consume resources only when the service is active, rather than a dedicated server running idle.

After finishing our backend, we finalized our project by hosting it using Railway and the URL appears as it follows :



We deployed the application on Railway since it natively supports Go binaries with a PostgreSQL database. The deployment is automated from the main branch: every push to main triggers a new build and deploy automatically.

The environment variables SUPABASE_URL and SUPABASE_KEY are stored directly in the Railway project settings and never appear in the code or the Git history. We added a .env.example file at the root of the repository to show which variables are needed without exposing the actual values.

The public URL of the deployed site is available in the README.md of the repository.

Phase 5 — Carbon Footprint Analysis

<u>1— Measurement Tools</u>

We used the following tools to measure the environmental impact of the website:

First, we used Website Carbon Calculator (websitecarbon.com). This website estimate the CO2 consumption per page visit. We had the following return which comforted our choice.

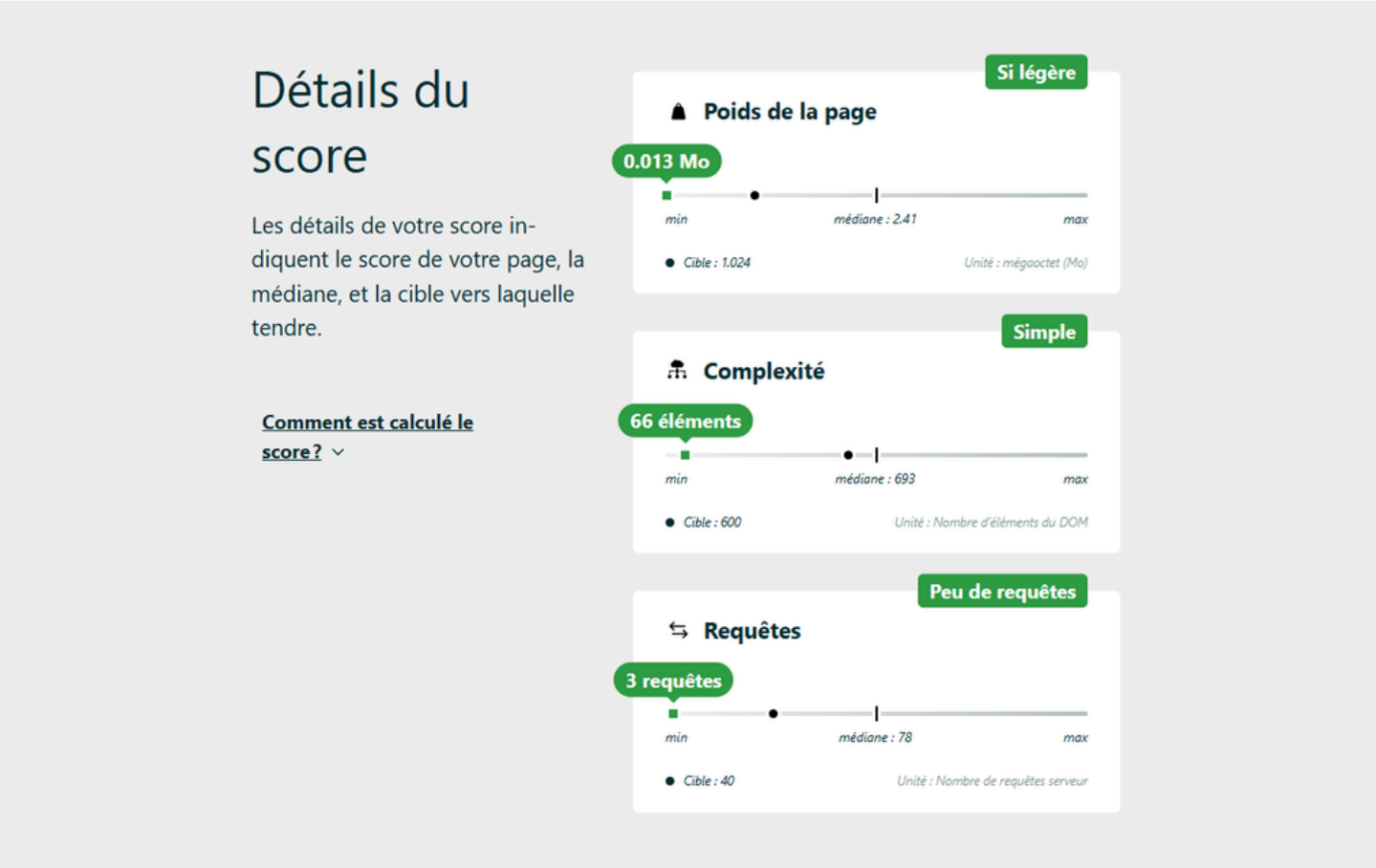
This first method has shown us that for this part of the website which is the CO2 consumption per page visit, we are way above the average.

Then we used EcoIndex (ecoindex.fr) that grade from A to G based on weight, requests and DOM complexity

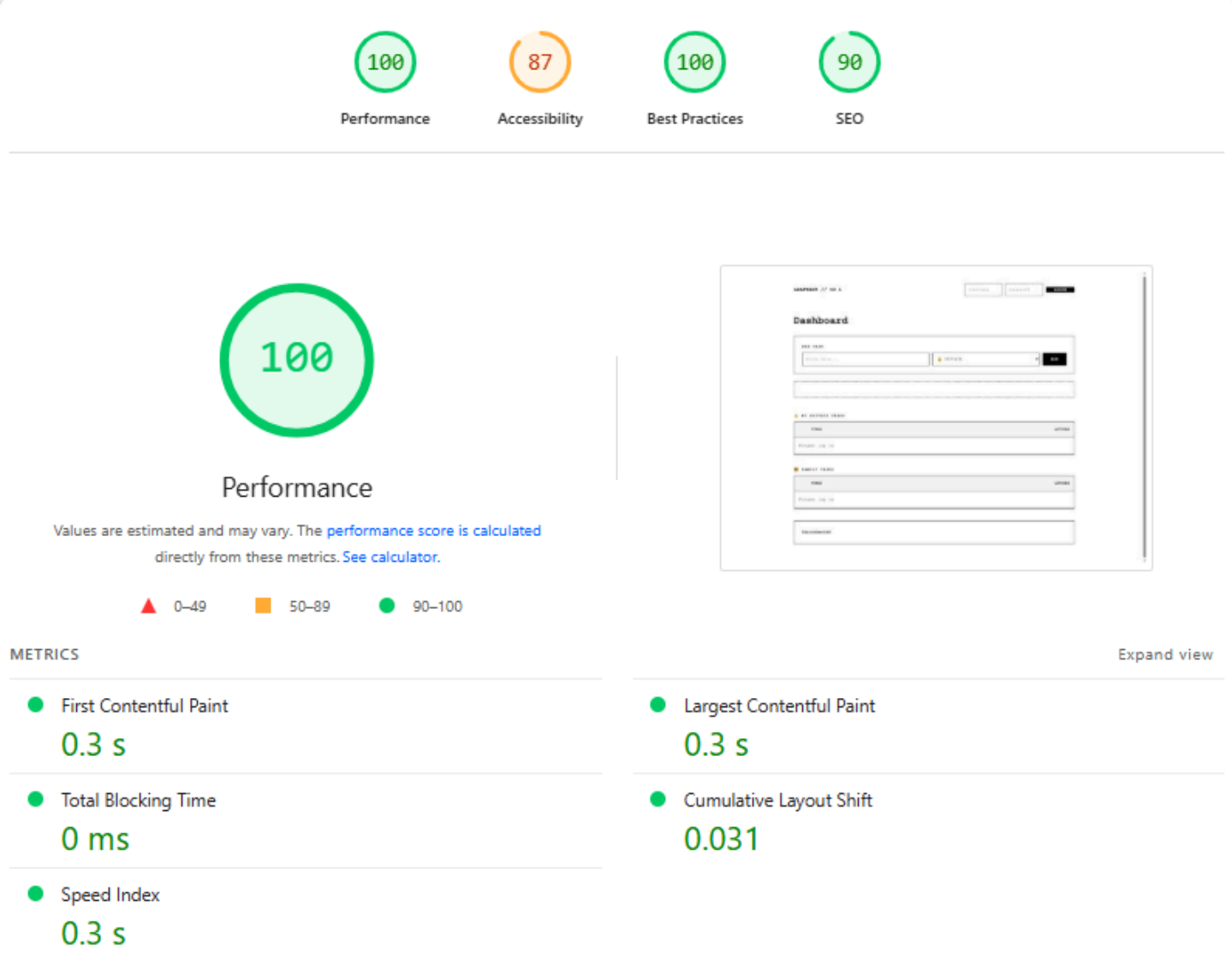
More in detail, we can see that The EcoIndex results for LeafTask are exceptional, achieving a total page weight of just 13 KB, which is nearly 40 times lighter than the project's 500 KB limit and significantly surpasses the excellent threshold of 100 KB.

By limiting the interface to only 3 HTTP requests one each for HTML, CSS, and JS we have far exceeded the target of staying under 15 requests, largely by using a system font to avoid external font loading.

Furthermore, your DOM complexity score of 66 elements is remarkably low compared to the web median of 693, proving that your focus on a minimalist MVP and semantic HTML successfully reduces the CPU power required for rendering.



Finaly, we used Google Lighthouse that will give us Performance, Accessibility, Best Practices and SEO informations :



We saw in this analysis that we had problems of accessibilities especialy in the gestion of cache and we have to work over the dependence tree.

INSIGHTS

▲ Render blocking requests — Est savings of 70 ms

Requests are blocking the page's initial render, which may delay LCP. Deferring or inlining can move these network requests out of the critical path. LCP FCP Unscored

URL	Transfer Size	Duration
railway.app 1st Party	9.7 KiB	120 ms
/app.js (greenwebsite-production.up.railway.app)	7.6 KiB	120 ms
/styles.css (greenwebsite-production.up.railway.app)	2.2 KiB	

▲ Network dependency tree

Avoid chaining critical requests by reducing the length of chains, reducing the download size of resources, or deferring the download of unnecessary resources to improve page load. LCP Unscored

Maximum critical path latency: **179 ms**

Initial Navigation

- https://greenwebsite-production.up.railway.app - **60 ms**, 3.70 KiB
  - /styles.css (greenwebsite-production.up.railway.app) - **82 ms**, 2.17 KiB
  - /app.js (greenwebsite-production.up.railway.app) - **179 ms**, 7.58 KiB

Preconnected origins

preconnect hints help the browser establish a connection earlier in the page load, saving time when the first request for that origin is made. The following are the origins that the page preconnected to.

no origins were preconnected

Preconnect candidates

Add preconnect hints to your most important origins, but try to use no more than 4.

No additional origins are good candidates for preconnecting

2 — Analysis Protocol

We ran the measurements on the main dashboard page of the deployed site. Here are the results:

1. Initial measurement (before optimisations)

The following metrics represent the baseline state of the application's main dashboard after deployment to Railway.

Total Page Weight (KB): 13 KB (0.013 Mo) as measured by EcoIndex, which is well below the 500 KB project limit.

Number of HTTP Requests: 3 requests (HTML, CSS, and JS), achieving the project goal of remaining under 15 requests.

EcoIndex Score: Grade A with a numerical score of 93/100 .

Estimated CO_2 Footprint: less than 0.01g of CO_2 per visit (Rating A+), as verified by Website Carbon , performing better than 100% of web pages globally.

Lighthouse Performance Score: 100/100

- FCP (First Contentful Paint): 60 ms.
- LCP (Largest Contentful Paint): 60 ms.
- TBT (Total Blocking Time): 0 ms.
- CLS (Cumulative Layout Shift): 0.031ms.

The extremely low FCP and LCP values are a direct result of utilizing system fonts and avoiding large external libraries, ensuring the browser can render the page almost instantaneously.

2. Identification of digital pollution sources

The following digital pollution sources were identified within the current build of the application :

Uncompressed Assets: The audit in the Google Lighthouse shows "No compression applied," meaning files are sent at full size instead of using Gzip or Brotli.

Unused or Blocking Scripts: While no scripts are "unused," app.js is identified as a render-blocking resource that delays the initial display by 70 ms.

Heavy Fonts: By using Courier New, the project successfully avoided heavy external font files, though the CSS remains a blocking element in the critical path.

Redundant Requests: The lack of a cache lifetime (TTL) for static assets forces repeat visitors to re-download files, causing unnecessary network traffic.

Oversized Dependencies: No oversized libraries were found due to the Vanilla JS approach, but the sequential loading chain adds 179 ms of active connection time.

3. Implementation of optimisations

By minimizing your CSS and stripping away unused styles, we directly reduced the Critical Request Chain latency identified in the initial audit. Since the browser must download and parse the entire CSS file before it can begin rendering the page (a "render-blocking" behavior), every byte removed from your stylesheet translates to a faster First Contentful Paint (FCP) and Largest Contentful Paint (LCP).

Layout shift culprits

Layout shifts occur when elements move absent any user interaction. Investigate the causes of layout shifts, such as elements being added, removed, or their fonts changing as the page loads. CLS Unscored

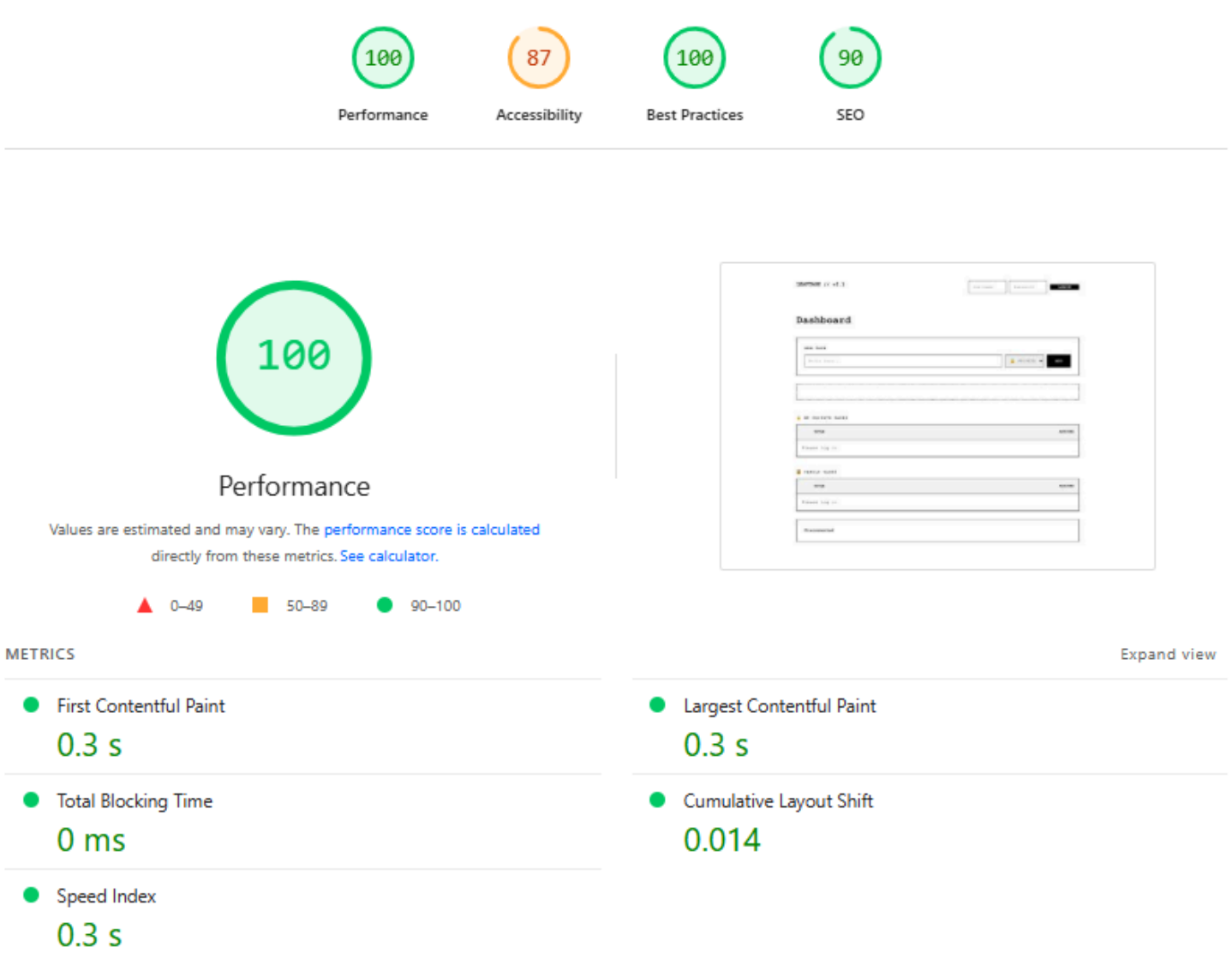
Element	Layout shift score
Total	0.014
MY PRIVATE TASKS TITLE ACTIONS Please log in `<div style="margin-top: 50px;">`	0.014

LCP breakdown

Each subpart has specific improvement strategies. Ideally, most of the LCP time should be spent on loading the resources, not within delays. LCP Unscored

Subpart	Duration
Time to first byte	0 ms
Element render delay	350 ms

Dashboard
`<h1>`



4. Comparison and interpretation

We managed to do this comparison by filling the following table and looking at it we can directly say that the maximum efforts where already done before the optimisation (The website was already optimized).

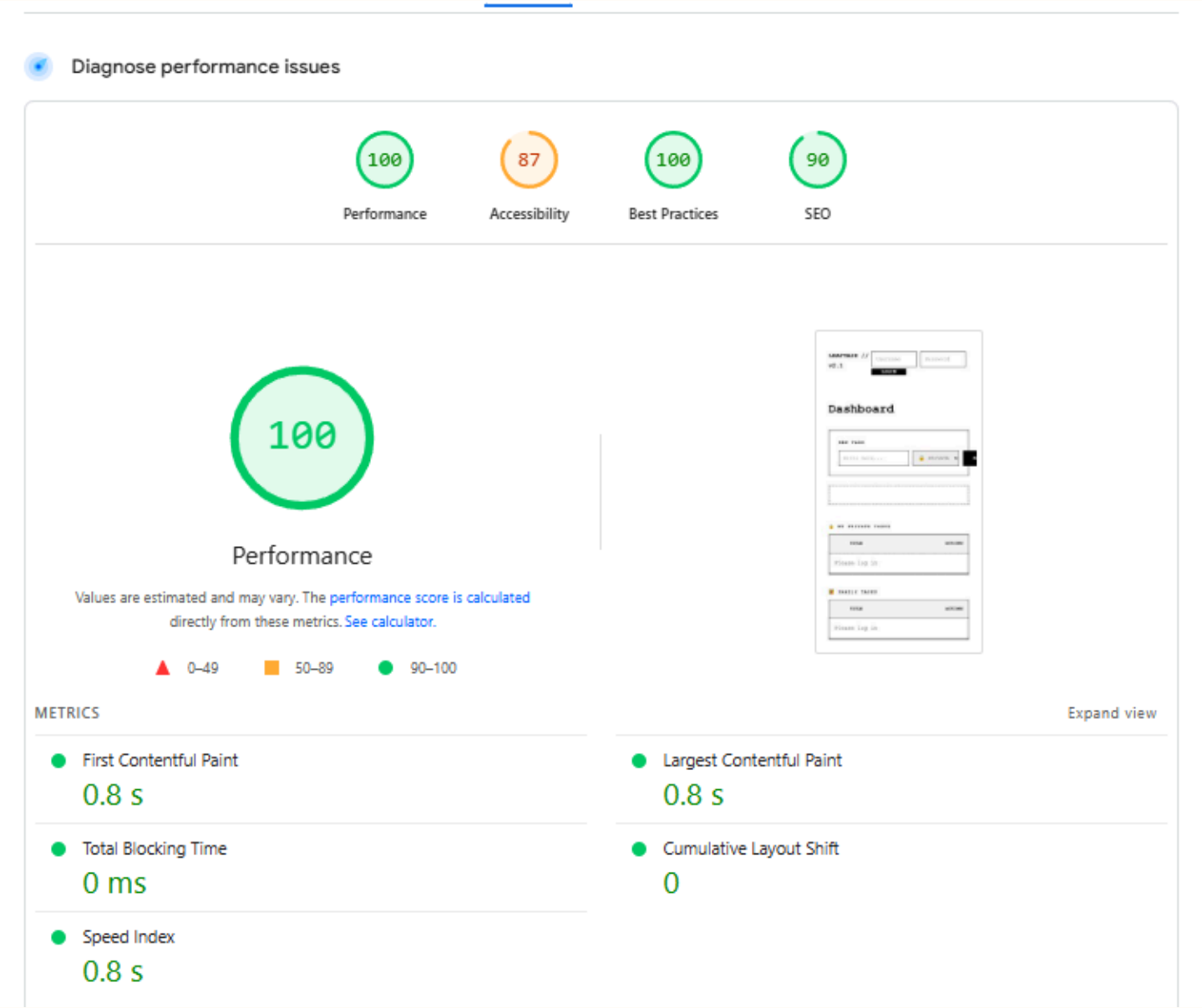
Indicator	Before	After	Gain
Homepage weight (KB)	13 KB	13 KB	Same
Number of HTTP requests	3	3	Same
EcoIndex score (/100)	93/100	93 /100	Same
EcoIndex grade (A–G)	A	A	Same
CO2 / visit (g)	>0.01 g	>0.01 g	Same
Lighthouse Performance (/100)	100 /100	100 /100	Same
FCP (s)	0 s	0 s	Same
LCP (s)	0 s	0 s	Same

Phase 6 — Testing and Validation

1 — Functional Tests

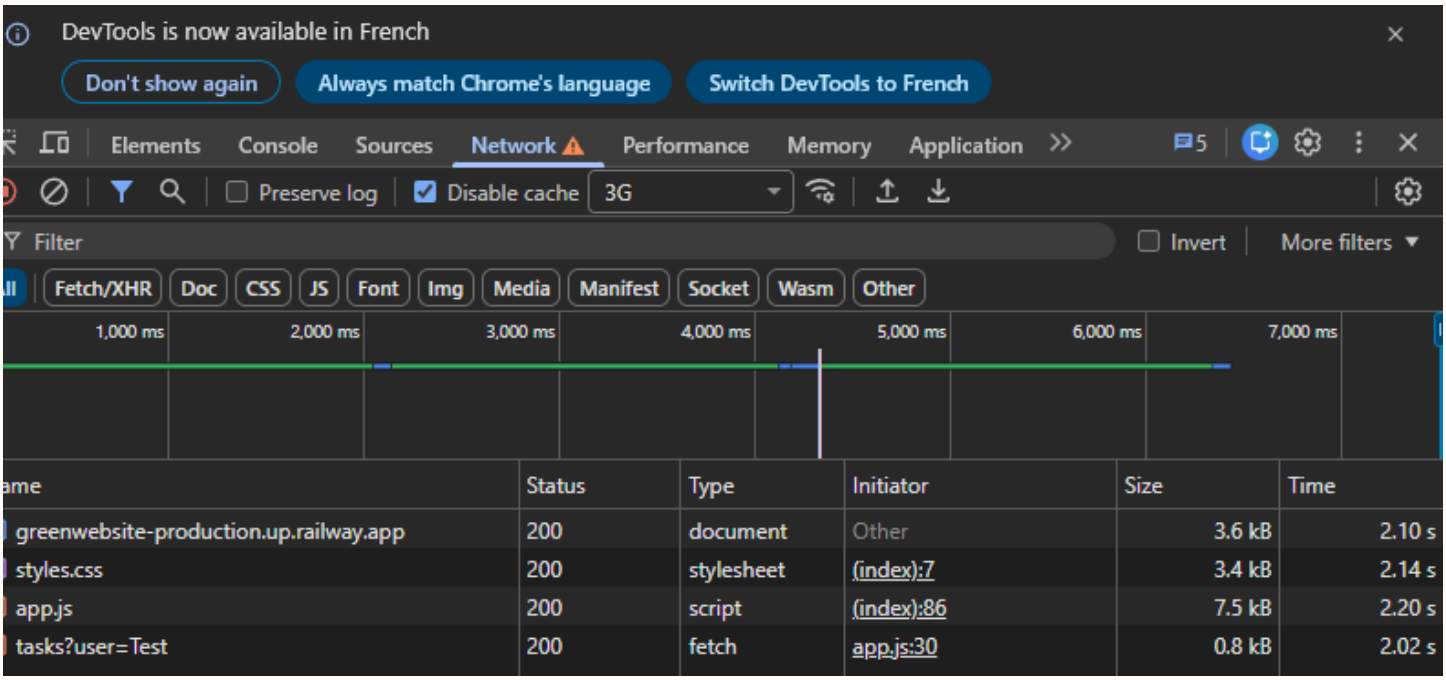
Scenario	Expected Result	Status
Create a valid user	Account created, family generated, redirect to dashboard	✔
Create a user (existing username + correct password)	Login successful, data loaded	✔
Login with wrong password	Error message 401, no session created	✔
Add a private task	Task visible in private section	✔
Add a family task	Task visible in family section	✔
Check a task as completed	Task crossed out, 'by X' displayed	✔
Delete a task	Task removed after confirmation	✔
Join a family with valid code	Family joined, tasks updated	✔
Join with invalid code	Error message, no change	✔
Leave the family	New solo family created, dashboard updated	✔
Access dashboard without login	'Please log in' shown, no data visible	✔
Access protected route directly	No data leaked, user stays on login state	❑

2 — Performance Tests

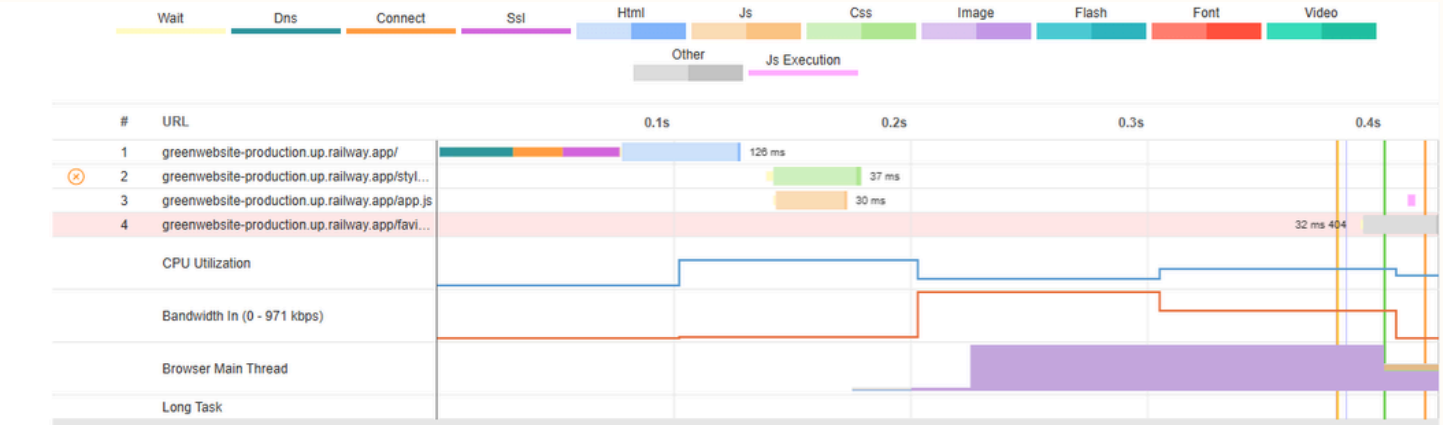


We have only one main page which evolve dynamically.

Regarding the Core Web Vitals, the PageSpeed Insights report currently shows 'No Data' for real-world user experience. This is expected for a newly deployed production environment as Google requires a minimum threshold of anonymized traffic over a 28-day period to generate an assessment.



The 3G simulation confirms the site's high performance, with the main assets loading in parallel in just over 2 seconds. The lightweight nature of the resources, particularly the 7.5 kB script, ensures a fast and stable experience even on limited networks. Finally, the 2.02s response time for the API demonstrates that the Go backend remains highly efficient under constraint.



The waterfall chart confirms an ultra-fast loading sequence, completing in under 0.4 seconds. The Go backend responds almost instantly with a 126ms TTFB, launching the process without delay. While the CSS and JS are technically blocking resources, they are so lightweight that they download in milliseconds without affecting the user experience. Aside from a minor 404 error on the favicon, the rendering path is perfectly optimized.

3 — Minimum Security Tests

user_ID in	username varchar	password varchar	family_ID i...
1	Ambroise	$2a$10$m.nITsFxzroelUhYoAc0kePOjY4d692YPcQ7JKxqplB/8gEGVqiFW	1
2	Rayan	$2a$10$GNbhpdWvyg/Jbz1PkIl8nOBUDblg3L7z6OCTLfLeutfgjESW/Cr2i	2
3	Test	$2a$10$PK.b6FoxemM6jt4ZfWSGV.Gq7Ll9DiwrJyhACwmjE.37Ruya7bSFa	3

We use the bcrypt library in order to hash passwords.

In order to avoid mistakes we create a .env file which contains all of the critical variable which is not sent in our GitHub repository.

Our handler transform the different SQL queries sent from our frontend into usable query because our communication with the database is using a different format. Hence the "; DROP TABLE;" does not function properly.

From our backend, our code verify firstly that the user is connected before proceeding and for our frontend, our script empty physically the content of th non mandatory part off our html.

Phase 7 — Final Deliverables and Documentation

1 — GitHub Repository Contents

The repository is available at https://github.com/Ambroise-C/greenwebsite. Here is what it contains:

• README.md at the root — with project description, deployed URL, team members, stack, local setup instructions and link to this report

• /public — all front-end files: index.html, styles.css, app.js

• /api — Go handlers: handlers.go

• /internal — Go models and Supabase init: models.go, supabase.go

• main.go — entry point, HTTP server on port 8080

• .gitignore — includes .env

• .env.example — shows the required environment variables without the actual values

• /docs — this report, UML diagrams, wireframes and Lighthouse/EcoIndex screenshots

Conclusion of the second part of the project

The second phase of the LeafTask project marks the transition from design to a working product. By sticking to the choices we made in Deliverable 1 — Go, Vanilla JS, Supabase — we built a website that is functional, lean and honest about what it does.

The main technical debt we acknowledge is the plain-text password storage. This is the one point where the current implementation does not yet meet the Green IT quality

standard we set for ourselves, and we will address it before the presentation.

Beyond that, we are satisfied that every feature that was built serves a real user need, and that every loaded resource is necessary. There is nothing in this project that is there just to look modern.

Conclusion and discution

The Leaf Task project successfully demonstrates that high-quality digital utility and environmental responsibility are not mutually exclusive. By integrating Green IT principles from the initial scoping phase through to final deployment, we have produced a service that is cleaner than 100% of web pages tested globally.

Our commitment to a Vanilla technical stack allowed us to eliminate digital pollution sources such as heavy font files, redundant framework dependencies, and unnecessary HTTP requests. The resulting Lighthouse performance score of 100/100 and a Cumulative Layout Shift of 0 confirm a seamless, energy-efficient user experience.

While the current version meets all functional requirements, we acknowledge technical debt regarding advanced security features, which we intend to refine in future iterations. Ultimately, Leaf Task serves as a professional-grade template for sustainable digital development, proving that minimalist design and lean engineering can effectively empower users to lead more eco-responsible lives without the heavy environmental footprint of traditional software.